Méthodes hybrides pour la génération de l'ensemble des solutions efficaces d'un programme linéaire en nombres entiers multi-objectif, avec un critère de décision linéaire fractionnaire

Résumé

Soit $D = \{x \in \mathbb{Z}_+^n : Ax \leqslant b\}$ non vide et borné, et soient $Z_1, \ldots, Z_p$ des critères à maximiser, chacun un rapport de formes affines de dénominateur strictement positif sur D. Ce mémoire traite deux questions portant sur l'ensemble efficace $E(P_D)$: le *produire en entier*, et y maximiser un critère de décision linéaire fractionnaire Φ.

Les méthodes existantes pour la seconde question travaillent dans l'espace des *décisions*, en tronquant D par des coupes de Sylva–Crema. Chaque coupe étant une disjonction, elle coûte p variables binaires et $p + 1$ contraintes de type grand-M : chaque sous-problème est strictement plus gros que le précédent. Nous conservons la partie inexplorée dans l'espace des *critères*, sous forme d'une liste de boîtes : la troncature devient une soustraction de boîtes, chaque sous-problème est D augmenté de quelques contraintes linéaires ordinaires, et **le modèle ne grossit jamais**.

Quatre contributions en découlent. D'abord une caractérisation qui rend l'ensemble efficace complet accessible : **si** v **est non dominé, alors tout** $x \in D$ **tel que** $Z(x) = v$ **est efficace**, de sorte que $E(P_D)$ est exactement la réunion des tranches du front et qu'*aucun test d'efficacité n'est payé sur un point de tranche*. Ensuite, une tranche est D augmenté de p équations *linéaires*, y compris pour des critères fractionnaires ; résoudre ces équations élimine r variables avant toute énumération et vaut un facteur médian de 49. Ensuite une hybridation qui paie : une recherche locale de Pareto fournit des points efficaces vérifiés, placés dans la liste de boîtes sans sondage, atteignant la totalité de $E(P_D)$ à 1,36 fois la méthode exacte pure sur 120 instances. Enfin, un $E(P_D)$ explicite admet *n'importe quelle* fonction de décision calculable, là où les méthodes de l'espace des décisions exigent une forme de rapport pour construire leurs coupes.

Sur le problème commun, la recherche dans l'espace des critères l'emporte à chaque valeur de p, de 1,52 à $p = 2$ jusqu'à 14,57 à $p = 6$, avec des optima identiques des deux côtés.

Table des matières

1 Introduction **3**
- 1.1 Contexte . . . 3
- 1.2 Problème traité . . . 3
- 1.3 Objectifs et plan . . . 3

2 État de l'art **5**
- 2.1 Deux familles de méthodes . . . 5
- 2.2 Positionnement de ce travail . . . 6

3 La méthode dans l'espace des critères **7**
- 3.1 D'où vient le coût de la méthode en espace des décisions . . . 7
- 3.2 La forme e_k, à valeurs entières . . . 7
- 3.3 Les boîtes, et pourquoi le modèle ne grossit pas . . . 7

4 Contributions **9**
- 4.1 L'ensemble efficace complet . . . 9
- 4.2 Une tranche est D plus p équations linéaires . . . 9
- 4.3 Ce qu'un $E(P_D)$ explicite autorise . . . 10
- 4.4 L'hybridation qui paie . . . 11

5 Exemples numériques illustrés **12**
- 5.1 Exemple A : l'instance publiée, $n = 2$ . . . 12
- 5.2 Exemple B : trois variables, toutes les branches exercées . . . 14
- 5.3 Ce que les deux exemples montrent ensemble . . . 16
- 5.4 L'hybridation, étape par étape . . . 16

6 Évaluation des performances **18**
- 6.1 Protocole de mesure . . . 18
- 6.2 Espace des décisions contre espace des critères . . . 18
- 6.3 L'hybridation . . . 19
- 6.4 Le domaine de fonctionnement . . . 19
- 6.5 Bilan des essais . . . 20

7 Validation **21**
- 7.1 Comment les résultats sont contrôlés . . . 21
- 7.2 Les hypothèses sont vérifiées, pas supposées . . . 21
- 7.3 Ce que la démarche a permis de corriger . . . 21

8 Conclusion et perspectives **23**
8.1 Ce qui a été obtenu 23
8.2 Perspectives 23

Bibliographie **25**

Chapitre 1

Introduction

1.1 Contexte

En optimisation multi-objectif en nombres entiers, il n'existe en général pas de solution unique, mais un *ensemble* de solutions efficaces, dont aucune n'est meilleure que les autres sur tous les critères à la fois. Le décideur doit alors trancher, et ce choix s'exprime souvent par un critère supplémentaire Φ à optimiser *sur* cet ensemble.

Le problème qui en résulte,

$$(P_E) \qquad \max\ \Phi(x) \quad \text{sous} \quad x \in E(P_D),$$

est difficile pour une raison structurelle : $E(P_D)$ est une réunion de faces de D, non convexe, et ne possède aucune description explicite par inégalités. On ne peut donc pas le passer à un solveur.

1.2 Problème traité

$$(P_D) \qquad \text{« max »}\ \ Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \qquad x \in D, \tag{1.1}$$

avec $D = \{x \in \mathbb{Z}_+^n : Ax \leqslant b\}$ non vide et borné, et chaque dénominateur $D_k(x) = d_k^\top x + b_k$ strictement positif sur D. Les deux hypothèses sont *vérifiées* par la mise en œuvre, pour une raison développée au chapitre 7 : les violer ne produit pas une erreur, mais une réponse fausse et d'apparence complète.

Définition 1.1. Un point $x \in D$ est *efficace* lorsqu'il n'existe aucun $y \in D$ vérifiant $Z(y) \geqslant Z(x)$ avec au moins une inégalité stricte. On note $E(P_D)$ l'ensemble des points efficaces et $F = Z(E(P_D))$ l'ensemble non dominé.

Remarque 1.2. Φ est une fonction de x et **non** de $Z(x)$. Deux points efficaces de même vecteur critère peuvent avoir des valeurs de Φ différentes. Toute construction menée dans l'espace des critères doit en tenir compte, et c'est la raison pour laquelle le transfert des méthodes de l'espace des critères du cas linéaire au cas fractionnaire n'est pas immédiat.

1.3 Objectifs et plan

Le travail poursuit les objectifs fixés dans le projet de mémoire :

1. concevoir des méthodes *hybrides* — exacte–métaheuristique et exacte–exacte — pour l'optimisation d'une fonction linéaire fractionnaire sur l'ensemble efficace entier ;
2. produire *un ensemble de solutions efficaces* engendré par ces algorithmes ;
3. **évaluer les performances** des algorithmes proposés par rapport aux méthodes existantes.

Le chapitre 2 situe le travail dans la littérature. Le chapitre 3 présente la méthode dans l'espace des critères. Le chapitre 4 expose les contributions. Le chapitre 6 donne l'évaluation des performances, et le chapitre 7 la validation.

Chapitre 2

État de l'art

2.1 Deux familles de méthodes

Les méthodes exactes pour l'optimisation sur l'ensemble efficace se répartissent en deux familles, selon l'espace dans lequel elles cherchent.

L'espace des décisions. La région non explorée est maintenue dans l'espace des variables, et réduite par des coupes. La coupe de Sylva–Crema [13] supprime $\{x : Z(x) \leqslant Z(\tilde{x})\}$; comme il s'agit d'une disjonction, elle s'exprime par p variables binaires et $p+1$ contraintes de type grand-M, et le test d'efficacité employé est celui d'Ecker et Kouada [7]. Toute la littérature sur un critère *fractionnaire* au-dessus d'un ensemble efficace entier appartient à cette famille : Drici, Ouaïl et Moulaï [6], Mahdi et Chaabane [9], Ouaïl et Chergui pour le cas hyperbolique [11], Chaiblaine, Moulaï et Cherfaoui pour deux fonctions fractionnaires [4], Mekhilef et Moulaï pour un objectif quadratique [10], ainsi que la méthode dont cette mise en œuvre est issue [14].

L'espace des critères. La région non explorée est maintenue dans l'espace des valeurs des critères, sous forme de régions géométriques. Boland, Charkhgard et Savelsbergh [3] travaillent dans l'espace des critères de dimension pleine et le décomposent à chaque itération ; leurs méthodes *balanced box* [1] et *triangle splitting* [2] en sont les précurseurs bi-objectif. L'énumération de l'ensemble non dominé tout entier relève du même espace : la récursion ε-contrainte d'Özlen et Azizoğlu [12] et les algorithmes d'exclusion de régions de Lokman et Köksalan [8] en sont deux approches générales. Les synthèses rapportent cette famille comme actuellement la plus efficace — **mais pour un critère de décision *linéaire***.

Une classe voisine, à lire avant toute revendication. Cherfaoui et Moulaï [5] engendrent l'ensemble efficace de programmes en nombres entiers *linéaires plus linéaires fractionnaires*, dont les critères sont une *somme* $c^\top x + (p^\top x + \alpha)/(q^\top x + \beta)$ et non un rapport. En chassant le dénominateur au niveau v, la partie quadratique $(c^\top x)(q^\top x)$ subsiste : l'ensemble de niveau est une *conique*, non un hyperplan. La caractérisation de la section 4.3 prédit donc que les deux travaux portent sur des classes de critères *disjointes*. C'est une prédiction tirée de l'énoncé de la classe, non une lecture de l'article, qui n'a pas pu être consulté.

2.2 Positionnement de ce travail

Deux éléments de la construction présentée ici relèvent de l'état de l'art et ne sont pas revendiqués : la **décomposition de l'espace des critères**, établie par les travaux ci-dessus [3, 1, 2] pour un objectif linéaire, et le **schéma d'hybridation** métaheuristique–exacte, dont la taxonomie est connue et dont la variante employée ici — produire une approximation du front par une métaheuristique, puis la faire raffiner par une méthode exacte — est la forme standard.

Ce que la littérature fractionnaire ne semble pas faire, c'est travailler dans l'espace des critères. La combinaison des deux, avec les éléments techniques qui la rendent *exacte* plutôt qu'approchée, constitue l'apport de ce mémoire :

— la forme e_k à valeurs entières (section 3.2), qui remplace l'estimation d'un pas minimal par un seuil exact ;

— les résultats de tranche du chapitre 4, qui donnent l'ensemble efficace complet ;

— la classe de fonctions de décision admissibles que cet ensemble explicite ouvre (section 4.3).

Remarque 2.1. Cette section indique ce qui a été trouvé et ce qui ne l'a pas été ; elle ne constitue pas une revendication de nouveauté. Une revue bibliographique complète, incluant la lecture intégrale des sources, reste à mener.

Chapitre 3

La méthode dans l'espace des critères

3.1 D'où vient le coût de la méthode en espace des décisions

Chaque coupe de Sylva–Crema ajoute p variables binaires et $p+1$ contraintes au modèle. Le coût est donc *structurel* et non accidentel : sur une instance mesurée, le modèle passe de 5 colonnes et 7 lignes à 95 et 127 au fil de 30 coupes, et le profil montre que 100% du temps est passé dans le programme en nombres entiers — la construction des coupes et la réparation ne comptent pour rien. Rien en dehors des sous-problèmes ne mérite d'être optimisé, et c'est précisément leur croissance que la méthode en espace des critères n'a pas.

3.2 La forme e_k, à valeurs entières

Les données de chaque critère sont mises à l'échelle entière (numérateur et dénominateur multipliés par le même facteur, le rapport est inchangé). On pose alors

$$e_k(x;\bar{x}) \;=\; D_k(\bar{x})\,D_k(x)\,\big(Z_k(x)-Z_k(\bar{x})\big), \tag{3.1}$$

qui est **linéaire en x et à valeurs entières**, de sorte que

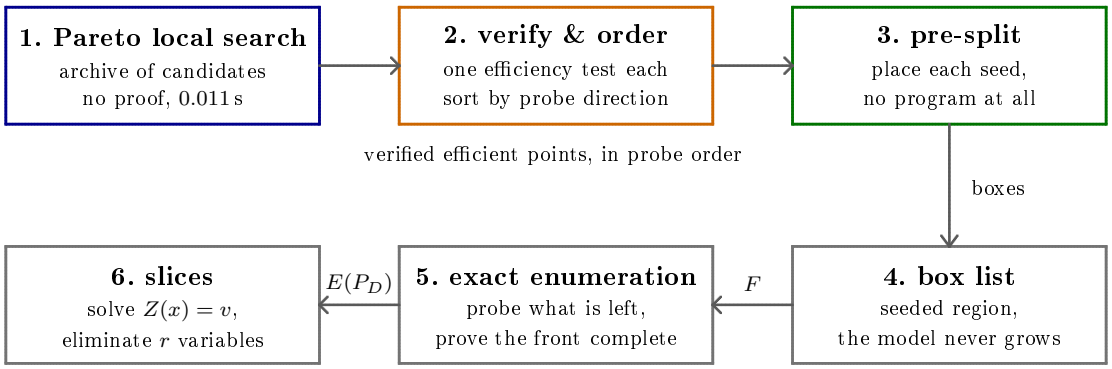
$$Z_k(x) > Z_k(\bar{x}) \iff e_k(x;\bar{x}) \geqslant 1. \tag{3.2}$$

C'est le point qui rend le passage des critères linéaires aux critères fractionnaires *fidèle* et non approché : aucun pas minimal n'a à être estimé. Les deux dénominateurs étant strictement positifs sur D, multiplier par eux ne change ni le sens ni la validité de l'inégalité.

3.3 Les boîtes, et pourquoi le modèle ne grossit pas

La partie inexplorée est conservée comme une liste de boîtes dans l'espace des critères. Tronquer revient à soustraire une boîte, et chaque sous-problème est D augmenté de quelques lignes linéaires ordinaires issues de (3.2). Aucune variable binaire, aucun grand-M, aucune croissance d'une itération à la suivante.

La partition est disjointe : l'enfant k reçoit $e_k \geqslant 1$ et $e_j \leqslant 0$ pour $j<k$, ce qui couvre la boîte privée du centre sans recouvrement.


1. Pareto local search
archive of candidates
no proof, 0.011 s
2. verify & order
one efficiency test each
sort by probe direction
3. pre-split
place each seed,
no program at all
verified efficient points, in probe order
boxes
6. slices
solve $Z(x) = v$,
eliminate r variables
$E(P_D)$
5. exact enumeration
probe what is left,
prove the front complete
F
4. box list
seeded region,
the model never grows

Chapitre 4

Contributions

4.1 L'ensemble efficace complet

L'énumération du front prouve la complétude de sa liste de *vecteurs*, mais un seul point est conservé par tranche : la liste de *points* n'est pas l'ensemble efficace. Cet écart se comble pour presque rien.

Théorème 4.1 (Caractérisation par tranches)**.** *Si v est non dominé, alors* ***tout*** $x \in D$ *tel que $Z(x) = v$ est efficace.*

Démonstration. Supposons qu'un $y \in D$ domine un tel x. Alors $Z(y) \geqslant Z(x) = v$ avec au moins une inégalité stricte, ce qui signifie que v est dominé, contrairement à l'hypothèse. Donc aucun y ne domine x, et x est efficace. □

Corollaire 4.2. *$E(P_D) = \{x \in D : Z(x) = v$ pour un v non dominé$\}$, et* ***aucun test d'efficacité n'est payé sur un point de tranche.***

L'opération la plus coûteuse de la méthode n'est tout simplement pas invoquée. La démonstration n'utilise aucune propriété de Z — ni linéarité, ni continuité, ni forme de rapport — de sorte que la caractérisation vaut pour des critères arbitraires. Ce qui exige une structure, c'est seulement l'*énumération* de chaque tranche.

4.2 Une tranche est D plus p équations linéaires

Fixer $Z_k(x) = v_k$ est linéaire en x *même pour un critère fractionnaire* : comme $d_k^\top x + b_k > 0$ sur D, l'équation $(c_k^\top x + a_k)/(d_k^\top x + b_k) = v_k$ se libère en

$$(c_k - v_k d_k)^\top x \;=\; v_k b_k - a_k. \tag{4.1}$$

Aucun binaire, aucun grand-M, rien qui croisse d'une tranche à la suivante : la même propriété que celle de la recherche par boîtes, obtenue une seconde fois.

Résoudre ces équations plutôt que s'en servir comme coupes. L'élimination de Gauss en arithmétique rationnelle exacte exprime r variables en fonction des autres ; la substitution laisse un système équivalent à $n - r$ inconnues, et le balayage porte sur celui-là. L'espace de recherche passe de $(ub + 1)^n$ à $(ub + 1)^{n-r}$.
L'exactitude n'est pas affectée : tous les coefficients sont des rationnels exacts et l'élimination l'est aussi, de sorte qu'il s'agit d'un changement d'ordre de parcours et non d'une approximation.

TABLE 4.1 – Élagage sur les équations de tranche contre résolution de ces équations. Quatorze instances, $n = 7 \dots 12$; ensembles de points identiques dans tous les cas.

instance	élagage	élimination	rapport
$n = 9$, $p = 3$	29,71 s	0,31 s	96,7×
$n = 10$, $p = 5$	119,47 s	0,92 s	129,9×
$n = 12$, $p = 3$	796,84 s	8,34 s	95,5×
médiane des quatorze			**49,2×**

4.3 Ce qu'un $E(P_D)$ explicite autorise

C'est la contribution que l'on défendrait en premier, parce qu'elle ne repose pas sur le fait d'être le plus rapide : elle dit quels problèmes peuvent être *posés*.

> Une méthode en espace des décisions exige une *structure dans* Φ et rien de $E(P_D)$; cette voie exige $E(P_D)$ et rien de Φ.

Théorème 4.3 (Classe admissible)**.** *Soit $E(P_D)$ connu et prouvé complet. Alors pour* toute *fonction f définie sur D,* $\max\{f(x) : x \in E(P_D)\}$ *et l'ensemble complet de ses maximiseurs se calculent exactement en évaluant f sur $E(P_D)$. Aucune linéarité, monotonie, continuité, dérivabilité ni forme de rapport n'est requise.*

La démonstration est la finitude de $E(P_D)$ et rien d'autre. Le contenu n'est pas dans l'énoncé : il est dans le fait d'avoir payé $E(P_D)$ à un prix qui rend ce paiement intéressant, et l'énoncé est ce que ce prix achète.

TABLE 4.2 – Six formes de fonction de décision sur l'instance publiée, chacune vérifiée contre une maximisation exhaustive de référence. Φ_1, Φ_2 sont linéaires fractionnaires.

fonction de décision	admise par un branch-and-cut ?	optimum
Φ_1	oui — le cas classique	5/17
$\Phi_1 + \Phi_2$	non	47/35
$\max(\Phi_1, \Phi_2)$	non	6/5
$\Phi_1 \cdot \Phi_2$	non	6/35
$(x_1^2 + 1)/(x_2^2 + 1)$	non	10
définie par morceaux	non	3/11

Un branch-and-cut pour un Φ linéaire fractionnaire tire ses coupes et ses bornes de la forme de rapport. Remplacer Φ par une *somme* de deux rapports ne le ralentit pas : il cesse de s'appliquer. Ici, ce cas coûte exactement ce que coûte un rapport unique.

Plusieurs fonctions de décision à la fois. Optimiser *deux* fonctions linéaires fractionnaires sur un ensemble efficace entier constitue un problème à part entière dans la littérature, avec son propre branch-and-bound. Ici, c'est un filtre de dominance sur un ensemble déjà en main, et le coût ne dépend ni du nombre de fonctions ni de leur nature. Sur l'instance publiée, le filtre à deux rapports conserve 4 des 7 points efficaces, exactement la réponse exhaustive.

4.4 L'hybridation qui paie

Une recherche locale de Pareto avec archive fournit des points efficaces **vérifiés**, placés dans la liste de boîtes **sans sondage**. La correction se situe là : l'archive est filtrée par dominance *parmi ce qu'elle a vu*, ce qui n'est pas le test exact ; rien ne franchit la frontière sans vérification. La métaheuristique décide *où chercher* ; elle ne décide jamais de ce qui est vrai.

L'ordre est toute la différence. C'est le résultat pratique le plus transférable du travail. L'hybridation ne paie que si les graines sont transmises dans l'ordre où le sondage de la méthode exacte les aurait rencontrées.

TABLE 4.3 – Sondages contre l'énumération non amorcée, selon l'ordre de transmission. Vingt-sept instances, aucune censurée ; le nombre de sondages est déterministe.

ordre	sondages	IQR	a augmenté le compte sur
ordre de l'archive	0,83×	[0,74, 0,92]	2 sur 27
ordre du sondage	**0,69**×	[0,67, 0,75]	**aucune**
inversé (témoin)	0,90×	[0,83, 1,05]	9 sur 27

Le bras inversé est le témoin : il est *tout aussi trié* et ne diffère de l'ordre du sondage que par la direction. Or l'ordre du sondage n'augmente le nombre de sondages sur aucune des vingt-sept instances, tandis que l'inversé l'augmente sur neuf, et sa médiane se place avec l'ordre de l'archive. Ce que l'amorçage exploite est donc la *direction propre de la recherche*, et non le fait qu'une liste soit triée.

Et plus précisément : la première graine. La pré-partition consomme les graines *dans l'ordre*, de sorte que la première fixe la structure de boîtes de premier niveau. En conditionnant sur elle, sur 228 ordres et 26 instances : lorsque la première graine est celle que le sondage aurait rencontrée en premier, la médiane est 0,708 ; sinon 0,833. Cet écart représente l'essentiel de l'écart entre les deux ordres nommés.

Chapitre 5

Exemples numériques illustrés

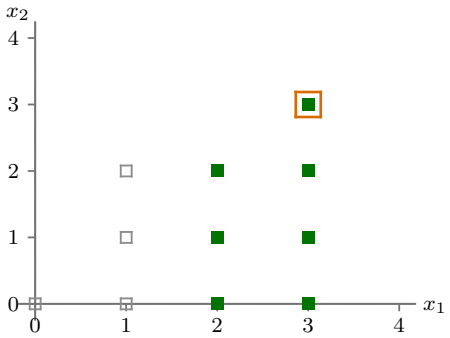
Les deux exemples de ce chapitre sont de taille volontairement réduite, afin que chaque étape soit vérifiable à la main. Ils montrent deux choses qu'un tableau ne montre pas : que le maximiseur sans contrainte de Φ est dominé, et que l'optimum est atteint par un chemin qu'une mise en œuvre naïve manquerait.

5.1 Exemple A : l'instance publiée, $n = 2$

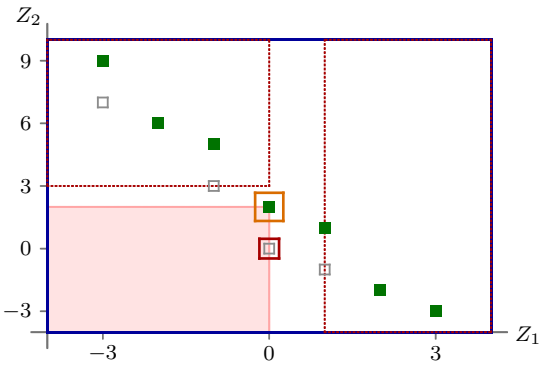
$$D = \{x \in \mathbb{Z}_+^2 : \ -2x_1 + x_2 \leqslant 0,\ 6x_1 + x_2 \leqslant 21,\ -2x_1 + 4x_2 \leqslant 6\}$$

avec $Z(x) = (x_1 - 2x_2,\ -x_1 + 4x_2)$ et $\Phi(x) = (x_1 + x_2 - 1)/(5x_1 + x_2 - 1)$.

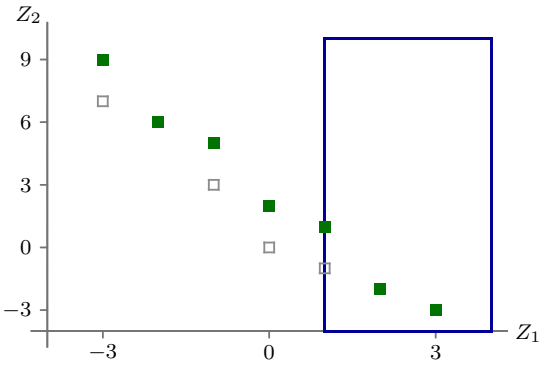
D compte 11 points, dont 7 efficaces, et la méthode conclut en 5 sous-problèmes. **L'optimum est** $\Phi^* = 5/17$ **en** $(3, 3)$, alors que le maximiseur libre de Φ est $(0, 0)$ avec $\Phi = 1$ — lequel est dominé. Le test d'efficacité n'est donc pas une formalité sur cette instance : il fait la différence entre une réponse et un nombre plausible mais faux.



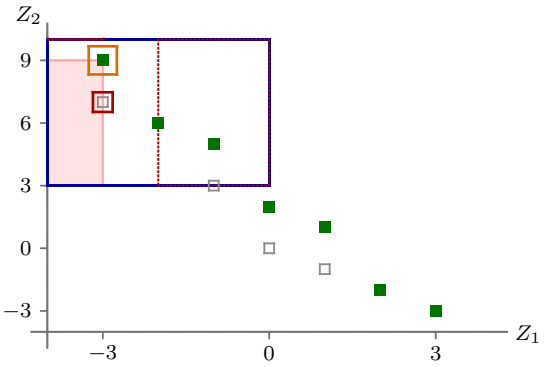
Étape 1 — la boîte racine



Étape 2 — une boîte éliminée sur sa borne

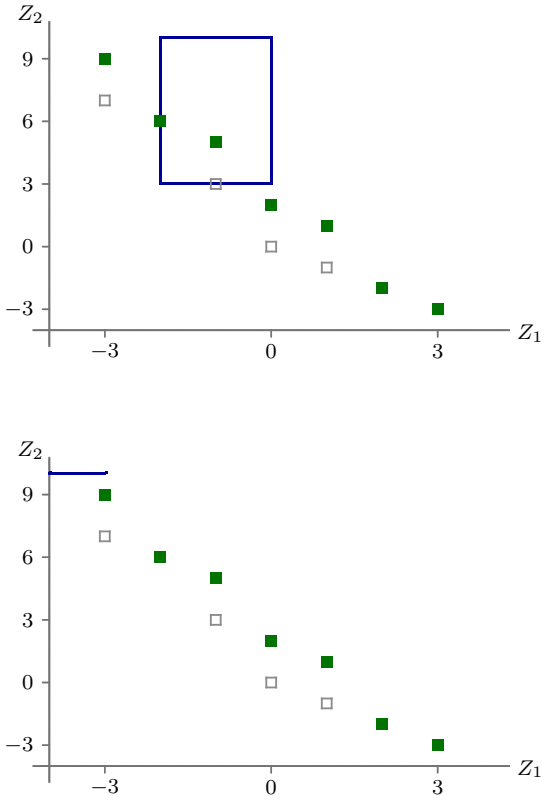


Étape 3 — l'optimum arrive, depuis un maximiseur dominé



C'est l'étape instructive. Le point $(3, 3)$ est trouvé comme centre *réparé* à partir du point dominé $(1, 2)$, et sa valeur provient du sous-problème Q — celui qui examine les points partageant le même vecteur critère. Une mise en œuvre qui omettrait Q, ce qui paraît raisonnable si l'on oublie la remarque 1.2, manquerait l'optimum.

Étapes 4 et 5 — le reste est la preuve

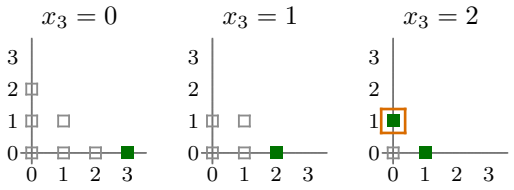


5.2 Exemple B : trois variables, toutes les branches exercées

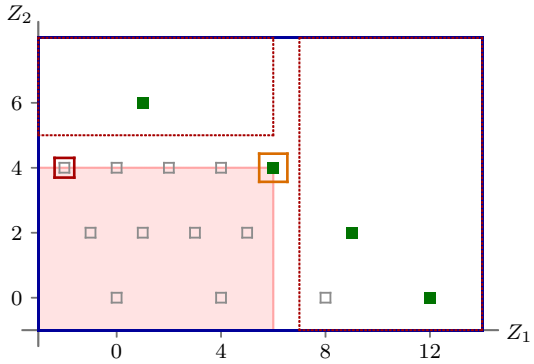
$$D = \{x \in \mathbb{Z}_+^3 :\ x_j \leqslant 3,\ x_1 + x_2 + 2x_3 \leqslant 5,\ 2x_1 + 3x_2 + x_3 \leqslant 6\}$$

avec $Z(x) = (4x_1 - x_2 + x_3,\ 2x_2 + 2x_3)$ et $\Phi(x) = (-2x_1 + 4x_2 - 3x_3 + 1)/(x_1 + 2x_2 + x_3 + 1)$.

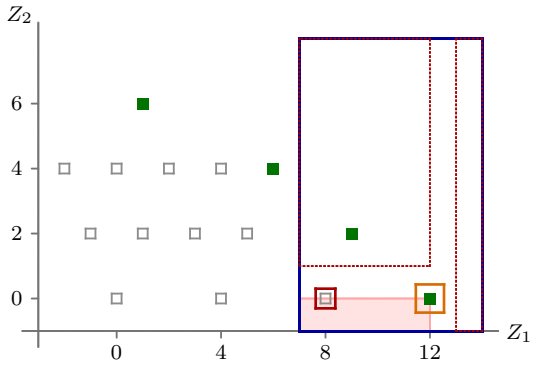
15 points admissibles, 4 efficaces, 4 sous-problèmes, $\Phi^* = -1/5$. Ici encore le maximiseur libre — $\Phi = 9/5$ en $(0, 2, 0)$ — est dominé.



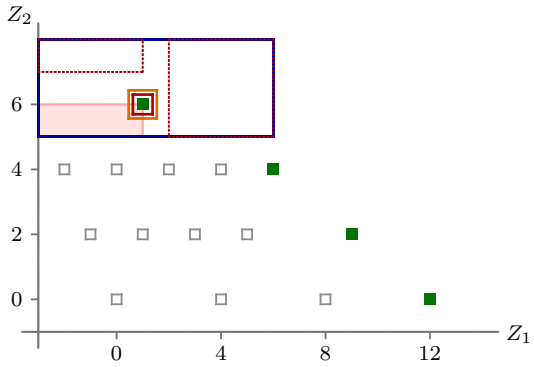
Étape 1 — réparation et premier incumbent



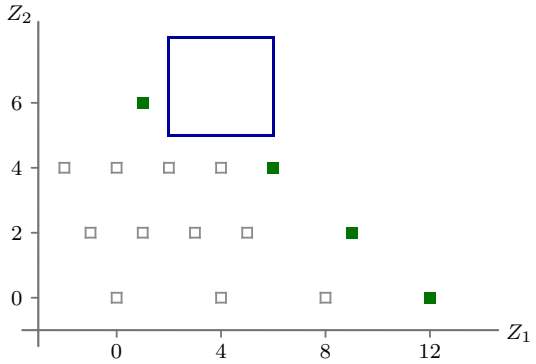
Étape 2 — une seconde réparation



Étape 3 — un maximiseur déjà efficace



Étape 4 — la sortie anticipée



La recherche s'arrête alors que des boîtes restent ouvertes : leur meilleure borne est inférieure à la valeur déjà prouvée, de sorte qu'aucune ne peut contenir mieux. C'est la sortie anticipée de la section 3.3, et elle laisse effectivement des boîtes non résolues tout en prouvant l'optimalité.

5.3 Ce que les deux exemples montrent ensemble

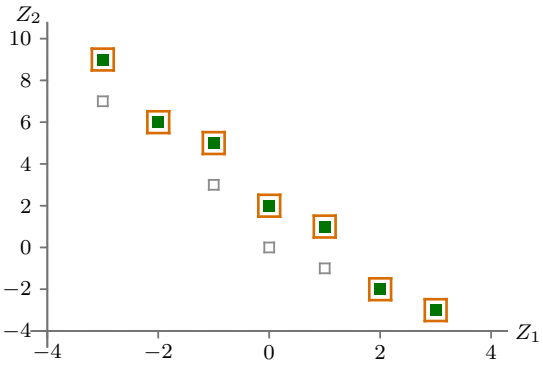
TABLE 5.1 – Les deux exécutions.

	n	$\lvert D\rvert$	$\lvert E(P_D)\rvert$	sous-problèmes	Φ^*
A (instance publiée)	2	11	7	5	5/17
B	3	15	4	4	$-1/5$

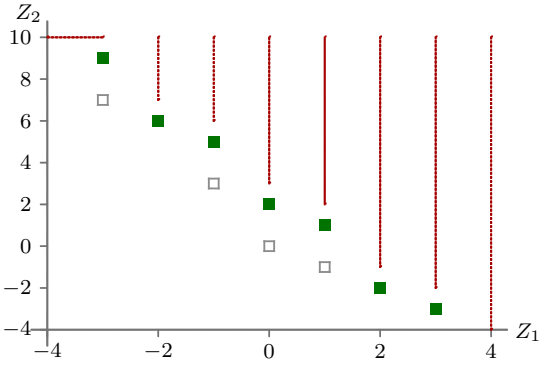
1. **Le maximiseur libre de Φ est dominé dans les deux cas.** $\Phi = 1$ en $(0, 0)$ contre une réponse de $5/17$; $\Phi = 9/5$ en $(0, 2, 0)$ contre $-1/5$.
2. **L'optimum passe par Q et non par un maximiseur**, dans A : une conséquence directe de la remarque 1.2.
3. **Aucune des deux exécutions n'énumère l'ensemble efficace** : cinq sous-problèmes pour sept points efficaces, quatre pour quatre. Sur les instances du chapitre 6 l'écart est l'essentiel : huit boîtes règlent une instance comptant 71 points efficaces parmi 629 admissibles.

5.4 L'hybridation, étape par étape

Les figures suivantes montrent ce que l'amorçage change : la marche de Pareto produit des points efficaces vérifiés, et la pré-partition les place dans la liste de boîtes sans payer de sondage.



À gauche, la recherche non amorcée ; à droite, l'état après transmission des graines dans l'ordre du sondage.



Chapitre 6

Évaluation des performances

Ce chapitre répond au quatrième point du projet : l'évaluation des performances des algorithmes proposés par rapport aux méthodes existantes.

6.1 Protocole de mesure

Toute l'arithmétique est rationnelle exacte, sans aucun flottant, de sorte que chaque comparaison oppose deux problèmes résolus exactement. La statistique est la **médiane des rapports par instance** avec l'écart interquartile, jamais un rapport de totaux — lequel serait décidé par la seule instance la plus lourde.

Ce qu'est la référence. Le côté « espace des décisions » de chaque comparaison est une **réimplémentation** de la méthode publiée, sur des instances engendrées ici — et non le code de ses auteurs ni leurs instances de référence. Les deux côtés partagent le même simplexe, la même arithmétique rationnelle exacte, le même branch-and-bound et la même machine, et la réimplémentation suit l'algorithme imprimé pas à pas. La comparaison *isole donc la différence algorithmique*, ce qui est une question plus propre qu'une comparaison de code à code — celle-ci mesurerait aussi le langage, le solveur sous-jacent et l'effort d'ingénierie.

En contrepartie, **les secondes absolues ne portent aucune information hors de cette mise en œuvre** : du Python pur sur des rationnels exacts est bien plus lent que tout solveur de production, également des deux côtés. Seuls les rapports se transportent, et seuls des rapports sont cités.

6.2 Espace des décisions contre espace des critères

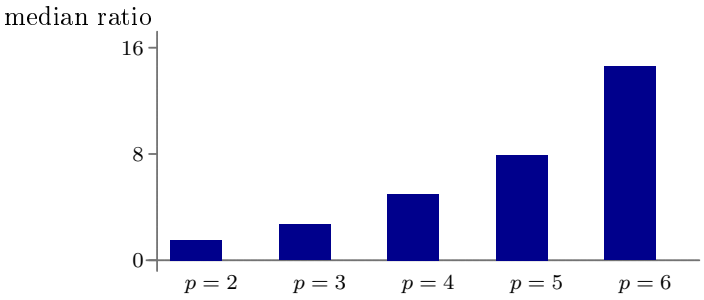


TABLE 6.1 – Le gain selon le nombre de critères, $n = 6$, **trente instances par ligne**.

p	décisions	critères	rapport (médiane)	IQR
2	0,07 s	0,04 s	1,52×	[1,25, 1,91]
3	0,34 s	0,12 s	2,69×	[1,87, 3,72]
4	0,50 s	0,13 s	4,99×	[2,39, 11,62]
5	2,19 s	0,25 s	7,93×	[3,61, 19,98]
6	3,67 s	0,31 s	**14,57**×	[7,90, 25,17]

La méthode l'emporte à chaque ligne, et l'avantage croît avec p — la direction qui importe, puisque c'est p qui rend ces problèmes difficiles. Trois des 150 exécutions ont épuisé un budget de 45 s et sont exclues plutôt que comptées à leur budget ; cette censure est *unilatérale* — elle ne peut que faire paraître la méthode en espace des décisions meilleure — de sorte que chaque marge citée est conservatrice. Toute exécution terminée a donné le même optimum des deux côtés.

La référence est-elle handicapée ? Le perdant a été profilé plutôt que simplement cité. Sur l'instance qui demande 137 s pour produire un front de 30 vecteurs en espace des décisions, 100% du temps est dans le programme en nombres entiers, et le modèle croît de 5 colonnes à 95. Le coût est structurel. Le seul handicap plausible — sonder en maximisant une direction plutôt qu'avec un objectif nul — a été testé : l'objectif nul est 4 à 13 fois *pire*, à coupes identiques.

6.3 L'hybridation

TABLE 6.2 – L'hybride contre la méthode exacte pure. Réponses identiques partout.

expérience	instances	rapport
$E(P_D)$ complet, $n = 6 \dots 12$	120	**1,36**×, plus rapide sur 116
critères véritablement fractionnaires	30	1,39×
cinq familles structurelles	150	1,23× médian, plus rapide sur 136

Chacune des cinq familles paie, entre 1,14× et 1,37× : l'avantage n'est donc pas un artefact d'une famille particulière.

6.4 Le domaine de fonctionnement

Deux quantités gouvernent le coût, et elles sont distinctes. L'énumération du front suit $|F|$; le complètement par tranches suit $n - p$, parce que les équations de tranche sont de rang p — mesuré par instance, et égal à p dans chaque cas — de sorte que l'élimination laisse $n - p$ variables libres. Le temps de tranche le confirme : 0,65, 2,09, 48,08 et 126,14 secondes à $n - p = 7, 9, 11, 13$.

À $p = 3$, la méthode est confortable jusqu'à $n = 12$ et résout encore des instances à $n = 14$–16. **Toutes les instances présentées dans ce mémoire se situent à** $n \leqslant 12$,

à l'intérieur de ce domaine avec de la marge : chaque chiffre rapporté provient donc du régime où la méthode fonctionne à l'écart des deux limites.

6.5 Bilan des essais

Vingt-deux modifications ont été essayées et mesurées. Les plus instructives :

TABLE 6.3 – Essais et verdicts.

essai	verdict	effet
espace des décisions $\to$ critères	**gain**	1,5× à 14,6×
amorçage par marche de Pareto	**gain**	**1,31**× vers tout $E(P_D)$
résolution des équations de tranche	**gain**	49× médian
tranches au lieu d'un test d'efficacité	**gain**	tout $E(P_D)$, sous 2×
dimensionnement de la marche	**gain**	1,00 $\to$ 1,53× à $n = 12$
hybride sur critères fractionnaires	**gain**	1,39×
Tchebychev comme générateur	**gain**	atteint les points non supportés
démarrage à chaud + point mémorisé	**gain**	20% de pivots en moins

Chapitre 7

Validation

7.1 Comment les résultats sont contrôlés

La mise en œuvre est en Python pur, sans aucune dépendance externe, en arithmétique rationnelle exacte de bout en bout — aucun flottant nulle part, donc aucune tolérance nulle part — et porte **112 tests** exécutés à chaque validation.

Plusieurs tests contrôlent des *propriétés* plutôt que des réponses, car une méthode qui perdrait silencieusement des points efficaces s'accorderait quand même avec l'optimum sur les instances favorables : la partition est disjointe ; la métaheuristique ne transmet jamais de point inefficace ; tout point mémorisé est efficace ; une racine démarrée à chaud atteint la même valeur que la racine froide ; la sortie anticipée laisse des boîtes ouvertes et prouve tout de même l'optimalité.

Les réponses sont recoupées de **quatre façons indépendantes** : l'énumération exhaustive, une énumération de l'ensemble non dominé par coupes répétées, un balayage trié avec tests de dominance, et un certificat d'optimalité qui règle 362 293 concurrents avec 3 tests d'efficacité exacts.

7.2 Les hypothèses sont vérifiées, pas supposées

Les deux hypothèses de (1.1) — D borné et dénominateurs strictement positifs — sont **vérifiées** par les points d'entrée, pour une raison précise : les violer ne produit pas une erreur mais une réponse fausse d'apparence complète. Sur un D non borné pourtant admissible, l'énumération rendait un *front vide marqué complet* ; sur un critère dont le dénominateur change de signe, elle rendait un seul vecteur comme front entier alors qu'un point admissible le domine.

Le coût de ces vérifications est une charge fixe par problème : un programme en nombres entiers pour la bornitude, un par critère *véritablement* fractionnaire pour les dénominateurs et aucun pour un critère linéaire, et n programmes mis en cache pour la boîte de variables. Face aux volumes de sous-problèmes du chapitre 6, ils ne se voient pas.

7.3 Ce que la démarche a permis de corriger

La discipline de mesure adoptée a permis de corriger des énoncés en cours de route, ce qui est rapporté ici parce que cela indique la fiabilité des chiffres conservés :

— la statistique est la médiane des rapports *par instance* — un rapport de totaux avait un jour été dominé par une seule instance fournissant 110,95 des 112,77 secondes ;

— le **travail déterministe** est compté à côté des secondes — sous-programmes, nœuds de branch-and-bound, pivots — car le plancher de bruit de la machine est de ±30% par instance et aucun nombre de répétitions ne le supprime ;

— une corrélation calculée sur des médianes par classe donnait −0,98 là où les paires brutes donnent −0,52 : c'est le chiffre brut qui est cité.

Chapitre 8

Conclusion et perspectives

8.1 Ce qui a été obtenu

1. **L'ensemble efficace complet** $E(P_D)$, et non le seul front, grâce au théorème 4.1 et sans payer de test d'efficacité sur un point de tranche. C'est ce que le projet attendait : un ensemble de solutions efficaces engendré par les algorithmes proposés — et il est livré *complet* là où le projet n'en demandait qu'un sous-ensemble.
2. **Une hybridation exacte–métaheuristique qui paie** : 1,36× sur 120 instances, plus rapide sur 116, 1,39× sur des critères fractionnaires, et un gain dans chacune des cinq familles.
3. **L'élimination des variables de tranche** : le plus grand gain unitaire du travail, 49× médian.
4. **La classe de décision admissible** (théorème 4.3) : un $E(P_D)$ explicite accepte toute fonction calculable, là où les méthodes existantes exigent une forme de rapport. C'est le résultat que l'on défendrait en premier, car il ne porte pas sur la vitesse mais sur les problèmes que l'on peut poser.
5. **L'évaluation des performances** demandée par le projet : de 1,52× à 14,57× selon p, avec une censure unilatérale en faveur de la méthode comparée et des optima identiques.

8.2 Perspectives

— **Comparaison avec les mises en œuvre des auteurs** sur leurs instances de référence. La comparaison menée ici isole la différence algorithmique ; celle-là mesurerait l'écart de bout en bout.

— **Élargissement du domaine de fonctionnement.** Les deux quantités qui le gouvernent étant identifiées — $|F|$ et $n - p$ — une réduction du coût de l'énumération du front élargirait directement le domaine.

— **Critères à ensembles de niveau non affines.** La machinerie de tranche s'étend à toute réunion finie de polyèdres, ce qui couvre déjà $|c^\top x + a|/(d^\top x + b)$ et les critères minimax ; l'énumération du front reste à étendre pour que ces classes soient traitées de bout en bout.

— **Revue bibliographique complète**, incluant la lecture intégrale des sources citées au chapitre 2. Les références ci-dessous ont été établies à partir d'index bibliographiques concordants, et non lues chez l'éditeur ; une entrée ([14]) reste à résoudre.

Bibliographie

[1] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. A criterion space search algorithm for biobjective integer programming : The balanced box method. *INFORMS Journal on Computing*, 27(4) :735–754, 2015.

[2] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. A criterion space search algorithm for biobjective mixed integer programming : The triangle splitting method. *INFORMS Journal on Computing*, 27(4) :597–618, 2015.

[3] Natashia Boland, Hadi Charkhgard, and Martin Savelsbergh. A new method for optimizing a linear function over the efficient set of a multiobjective integer program. *European Journal of Operational Research*, 260(3) :904–919, 2017.

[4] Yacine Chaiblaine, Mustapha Moulaï, and Y. Cherfaoui. An exact method for optimizing two linear fractional functions over the efficient set of a multiobjective integer linear fractional program. arXiv :2003.05364 [math.OC], 2020.

[5] Yasmine Cherfaoui and Mustapha Moulaï. Generating the efficient set of multiobjective integer linear plus linear fractional programming problems. *Annals of Operations Research*, 296(1) :735–753, 2021.

[6] Wassila Drici, Fatma Zohra Ouaïl, and Mustapha Moulaï. Optimizing a linear fractional function over the integer efficient set. *Annals of Operations Research*, 267(1) :135–151, 2018.

[7] J. G. Ecker and I. A. Kouada. Finding efficient points for linear multiple objective programs. *Mathematical Programming*, 8(1) :375–377, 1975.

[8] Banu Lokman and Murat Köksalan. Finding all nondominated points of multi-objective integer programs. *Journal of Global Optimization*, 57(2) :347–365, 2013.

[9] Sara Mahdi and Djamal Chaabane. A linear fractional optimization over an integer efficient set. *RAIRO-Operations Research*, 49(2) :265–278, 2015.

[10] Amal Mekhilef and Mustapha Moulaï. Quadratic optimization over a discrete Pareto set of a multi-objective linear fractional program. *Optimization*, 70(7) :1425–1442, 2021.

[11] Fatma Zohra Ouaïl and Mohamed El-Amine Chergui. Hyperbolic optimization over the integer efficient set of MOILFP. arXiv :1907.02036 [math.OC], 2019.

[12] Melih Özlen and Meral Azizoğlu. Multi-objective integer programming : A general approach for generating all non-dominated solutions. *European Journal of Operational Research*, 199(1) :25–35, 2009.

[13] John Sylva and Alejandro Crema. A method for finding the set of non-dominated vectors for multiple objective integer linear programs. *European Journal of Operational Research*, 158(1) :46–55, 2004.

[14] Leila Younsi-Abbaci. Optimizing a linear fractional function over an integer efficient set. *Reliability : Theory & Applications*, 2025.